

Tuple Introduction - Basic Operations and Examples

```
thistuple = ("this", "is", "a", "tuple")
print(thistuple)
print(len(thistuple))
print(type(thistuple))
print(thistuple[2])
print(thistuple[-1:0:-1])
print(thistuple[1::-1])
# Output
('this', 'is', 'a', 'tuple')
4
<class 'tuple'>
a
('tuple', 'a', 'is')
('is', 'this')
```

List Introduction - Basic Operations and Examples

Creating lists

```
print("=== Creating Lists ===")
numbers = [1, 2, 3, 4, 5]
fruits = ["apple", "banana", "cherry", "date"]
mixed_list = [1, "hello", 3.14, True, [1, 2, 3]]
empty_list = []
```

```
print(f"Numbers list: {numbers}")
print(f"Fruits list: {fruits}")
print(f"Mixed list: {mixed_list}")
print(f"Empty list: {empty_list}")
```

List length

```
print(f"\nLength of numbers list: {len(numbers)}")
print(f"Length of fruits list: {len(fruits)}")
```

```
print("\n=== Accessing Elements ===")
```

Accessing elements by index

```
print(f"First fruit: {fruits[0]}")
print(f"Last fruit: {fruits[-1]}")
print(f"Second number: {numbers[1]}")

# Slicing
print(f"First three numbers: {numbers[0:3]}")
print(f"Last two fruits: {fruits[-2:]}")
print(f"Every second number: {numbers[::2]}")

print("\n=== Adding Elements ===")
# Append - adds element at the end
fruits.append("elderberry")
print(f"After appending elderberry: {fruits}")

# Insert - adds element at specific position
fruits.insert(1, "apricot")
print(f"After inserting apricot at index 1: {fruits}")

# Extend - adds multiple elements
numbers.extend([6, 7, 8])
print(f"After extending numbers with [6, 7, 8]: {numbers}")

print("\n=== Removing Elements ===")
# Remove - removes first occurrence of value
fruits.remove("banana")
print(f"After removing banana: {fruits}")

# Pop - removes and returns element at index (default is last)
popped_fruit = fruits.pop()
print(f"Popped fruit: {popped_fruit}")
print(f"Fruits after popping: {fruits}")

popped_number = numbers.pop(0)
print(f"Popped first number: {popped_number}")
print(f"Numbers after popping first: {numbers}")

# Del - deletes element at index
```

```
del fruits[1]
print(f"After deleting fruit at index 1: {fruits}")

print("\n=== Modifying Elements ===")
# Changing elements
numbers[0] = 10
print(f"After changing first number to 10: {numbers}")

# Changing multiple elements
numbers[1:3] = [20, 30]
print(f"After changing numbers[1:3] to [20, 30]: {numbers}")

print("\n=== List Operations ===")
# Concatenation
list1 = [1, 2, 3]
list2 = [4, 5, 6]
combined = list1 + list2
print(f"Combining {list1} + {list2} = {combined}")

# Repetition
repeated = [0] * 5
print(f"Repeating [0] * 5 = {repeated}")

# Membership testing
print(f"Is 'apple' in fruits? {'apple' in fruits}")
print(f"Is 'orange' in fruits? {'orange' in fruits}")
print(f"Is 5 not in numbers? {5 not in numbers}")

print("\nList Methods")
test_numbers = [3, 1, 4, 1, 5, 9, 2, 6]
print(f"Original test numbers: {test_numbers}")

# Count occurrences
print(f"Count of 1 in test_numbers: {test_numbers.count(1)}")

# Find index
```

```
print(f"Index of first occurrence of 4:
{test_numbers.index(4)}")

# Sort (modifies original list)
test_numbers.sort()
print(f"After sorting: {test_numbers}")

# Reverse (modifies original list)
test_numbers.reverse()
print(f"After reversing: {test_numbers}")

# Clear - removes all elements
copy_list = test_numbers.copy()
copy_list.clear()
print(f"After clearing copy: {copy_list}")

print("\n=== List Comprehensions ===")
# Creating lists with comprehensions
squares = [x**2 for x in range(1, 6)]
print(f"Squares of 1-5: {squares}")

even_numbers = [x for x in range(1, 11) if x % 2 == 0]
print(f"Even numbers 1-10: {even_numbers}")

print("\n=== Nested Lists ===")
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(f"Matrix: {matrix}")
print(f"Element at row 1, column 2: {matrix[1][2]}")

print("\n=== Useful Built-in Functions ===")
sample_numbers = [10, 5, 8, 3, 7, 1, 9]
print(f"Sample numbers: {sample_numbers}")
print(f"Sum: {sum(sample_numbers)}")
print(f"Maximum: {max(sample_numbers)}")
print(f"Minimum: {min(sample_numbers)}")
```

```
print(f"Sorted (new list): {sorted(sample_numbers)}")
print(f"Original unchanged: {sample_numbers}")
```

=== Creating Lists ===

```
Numbers list: [1, 2, 3, 4, 5]
Fruits list: ['apple', 'banana', 'cherry', 'date']
Mixed list: [1, 'hello', 3.14, True, [1, 2, 3]]
Empty list: []
```

```
Length of numbers list: 5
Length of fruits list: 4
```

=== Accessing Elements ===

```
First fruit: apple
Last fruit: date
Second number: 2
First three numbers: [1, 2, 3]
Last two fruits: ['cherry', 'date']
Every second number: [1, 3, 5]
```

=== Adding Elements ===

```
After appending elderberry: ['apple', 'banana', 'cherry',
'date', 'elderberry']
After inserting apricot at index 1: ['apple', 'apricot',
'banana', 'cherry', 'date', 'elderberry']
After extending numbers with [6, 7, 8]: [1, 2, 3, 4, 5, 6, 7,
8]
```

=== Removing Elements ===

```
After removing banana: ['apple', 'apricot', 'cherry', 'date',
'elderberry']
Popped fruit: elderberry
Fruits after popping: ['apple', 'apricot', 'cherry', 'date']
Popped first number: 1
Numbers after popping first: [2, 3, 4, 5, 6, 7, 8]
After deleting fruit at index 1: ['apple', 'cherry', 'date']
```

=== Modifying Elements ===

```
After changing first number to 10: [10, 3, 4, 5, 6, 7, 8]
```

After changing numbers[1:3] to [20, 30]: [10, 20, 30, 5, 6, 7, 8]

=== List Operations ===

Combining [1, 2, 3] + [4, 5, 6] = [1, 2, 3, 4, 5, 6]

Repeating [0] * 5 = [0, 0, 0, 0, 0]

Is 'apple' in fruits? True

Is 'orange' in fruits? False

Is 5 not in numbers? False

List Methods

Original test numbers: [3, 1, 4, 1, 5, 9, 2, 6]

Count of 1 in test_numbers: 2

Index of first occurrence of 4: 2

After sorting: [1, 1, 2, 3, 4, 5, 6, 9]

After reversing: [9, 6, 5, 4, 3, 2, 1, 1]

After clearing copy: []

=== List Comprehensions ===

Squares of 1-5: [1, 4, 9, 16, 25]

Even numbers 1-10: [2, 4, 6, 8, 10]

=== Nested Lists ===

Matrix: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

Element at row 1, column 2: 6

=== Useful Built-in Functions ===

Sample numbers: [10, 5, 8, 3, 7, 1, 9]

Sum: 43

Maximum: 10

Minimum: 1

Sorted (new list): [1, 3, 5, 7, 8, 9, 10]

Original unchanged: [10, 5, 8, 3, 7, 1, 9]

dictionary operations and methods

```
def dictionary_intro():
```

```
    # Creating a dictionary
```

```
    my_dict = {
```

```
        "name": "Alibaba",
```

```
        "age": 30,
```

```

        "city": "New Delhi"
    }

    # Accessing values
    print(f"Name: {my_dict['name']}")
    print(f"Age: {my_dict['age']}")

    # Adding a new key-value pair
    my_dict["job"] = "Engineer"
    print(f"Updated Dictionary: {my_dict}")

    # Removing a key-value pair
    del my_dict["city"]
    print(f"After deletion: {my_dict}")

    # Checking if a key exists
    if "age" in my_dict:
        print("Age key exists in the dictionary.")
    else:
        print("Age key does not exist in the dictionary.")

    print("Iterating through dictionary:")
    for key, value in my_dict.items():
        print(f"{key}: {value}")

    # Getting keys and values
    keys = my_dict.keys()
    values = my_dict.values()
    print(f"Keys: {keys}")
    print(f"Values: {values}")

dictionary_intro()

```

Name: Alibaba

Age: 30

Updated Dictionary: {'name': 'Alibaba', 'age': 30, 'city': 'New Delhi', 'job': 'Engineer'}

After deletion: {'name': 'Alibaba', 'age': 30, 'job': 'Engineer'}

Age key exists in the dictionary.

Iterating through dictionary:

name: Alibaba

age: 30

job: Engineer

Keys: dict_keys(['name', 'age', 'job'])

Values: dict_values(['Alibaba', 30, 'Engineer'])